CognitiveMesh: An Adaptive Agent Architecture with Long-Term Memory and Self-Evolving Reasoning

Vispute Shreya Ganesh[1], Tanmay Rastogi[2], Priyadharsini M[3*], Karthik K[4]

[1,2,3,4] *School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India*

vispute.shreya2024@vitstudent.ac.in, tanmay.rastogi2024@vitstudent.ac.in, priyadharsini.m@vit.ac.in, k.karthik@vit.ac.in

Abstract

The recent technological advancement of the big language model (LLM) technology has enabled greatly enhanced capabilities for generative AI agents in terms of action, collaboration, and communication compared to earlier agents based on LLM technology. However, current systems still remain stateless in their functionalities during different sessions, and are largely reliant on pre-specified tools and fixed reasoning systems making them incapable of adapting to new situations. This academic publication proposes CognitiveMesh, which is a new framework for adaptive agents consisting of layered long-term memory and integration of tool invocation and synthesis mechanisms, and evolution of the reasoning system through continuous learning. Long-term memory in CognitiveMesh consist of episodic, semantic, and procedural components. The organization of memory along with retrieval and forgetting process is independently managed with a controller to be able to use knowledge during a few sessions. The module for tool learning enables retrieval of a number of tools and assist in their designing. Critique and Reflection unit allows transformation of unsuccessful experience into lightweight changes that allow for continuous improvement without full retraining of the model. CognitiveMesh has been tested through 3 benchmark tool. The improvements made by using this framework include increase in performance by 15.4% to 24.8%, while decreasing number of unnecessary usage of tools by 38%. In spite of introduction of a small lag, this framework displays consistent performance for long-term tasks and acquiring new tools.

Keywords: Generative AI agents, Large language models, Long-term memory, Tool learning, Self-evolving reasoning, Retrieval-augmented reasoning, Autonomous agents

1. Introduction

The design of Generative Agents is evolving rapidly. Advanced Large Language Models (LLMs) are enabling systems capable of multi-step planning and tool calling. Models such as GPT-3 and GPT-4 have demonstrated remarkable performance in executing long-running tasks (Brown et al., 2020; OpenAI, 2023). Many existing reasoning approaches allow language models to perform complex decomposition and reasoning (Wei et al., 2022; Yao et al., 2023). Advanced agent frameworks further utilize reasoning and actions, enabling agents to make inferences, combine them with evidence, and take effective decisions (Yao et al., 2023).

Despite significant achievements in the development of effective LLM agents, they still experience limitations when functioning over long periods of time. The vast majority of agents operate independently within each session, meaning that they do not effectively utilize knowledge acquired during previous interactions. Moreover, effective memory-assisted mechanisms for working with LLM agents are currently being developed (Packer et al., 2023; Lewis et al., 2020). However, typical vector-based retrieval systems do not provide a clear differentiation between temporary and durable aspects of interactions.

The utilization of technological instruments is another limitation. Tool-supported agents can perform a variety of actions using diverse functional mechanisms, such as APIs and external search systems (Schick et al., 2023). Technologies based on tool retrieval help agents identify the necessary tool from a variety of available instruments using semantic representations (Qin et al., 2023), while skill-based mechanisms enable agents to memorize successfully performed actions and reuse them in other situations (Wang et al., 2023). However, these methods remain restricted to the available tools and provide limited opportunities for developing new capabilities in the absence of appropriate mechanisms.

Another important aspect is self-improvement through the use of reflection. Agents that rely on reflection can assess unsuccessful actions and use the insights gained to improve the performance of subsequent activities (Shinn et al., 2023). Innovative techniques such as Low-Rank Adaptation (LoRA) provide another approach for improving a model without requiring complete retraining (Hu et al., 2022). These methods demonstrate the effectiveness of utilizing experience for improvement; however, there remains a challenge in creating an instrument capable of simultaneously utilizing memory, adaptive tools, and self-reflection.

Such limitations become crucial in long-term applications such as multi-week academic assistance, personal productivity

*Corresponding author: Priyadharsini M (priyadharsini.m@vit.ac.in)

management, and autonomous software maintenance. In these scenarios, an agent should be able to retain relevant experience across sessions, forget unnecessary information, select appropriate tools, and adapt its behavior when existing tools or approaches are inadequate. Previous studies have largely investigated memory, tool acquisition, and reflection as separate capabilities, motivating the development of a holistic approach that combines these abilities.

The provided paper describes *CognitiveMesh*, a holistic adaptive-agent framework consisting of three cooperating components: (1) a tiered long-term memory unit that differentiates episodic, semantic, and procedural knowledge; (2) a tool-creating unit that utilizes available tools and produces new ones when no adequate tools are available; and (3) a self-learning reasoning machine that learns from its own mistakes. These modules communicate through a dedicated integration layer, allowing memory to assist in tool selection, tool execution results to update memory, and reasoning processes to trigger memory retrieval or adaptation.

The main objectives of *CognitiveMesh* are to retain relevant context over a long period of time, provide additional capabilities through acquired and created tools, and improve reasoning through experience without retraining the underlying model. The framework has been evaluated using LTM-Bench, ToolBench-Ext, and WebArena-Lite, which explore long-term memory retention, tool usage, and real-world Internet operations, respectively.

The contributions of this research work can be summarized as follows:

1. The development of a multi-layered architecture for long-term memory that segregates knowledge into different memory types, including episodic, semantic, and procedural memory, while explicitly incorporating mechanisms for forgetting and consolidation. “‘
2. The introduction of a tool-learning framework that retrieves tools using embedding techniques and synthesizes new tools within sandboxed environments.
3. The creation of self-evolving reasoning techniques that combine immediate reflection with effective updating of low-rank policies.
4. The evaluation of the effectiveness of the proposed framework through experiments conducted across three benchmark suites. “‘

The remainder of this paper is organized as follows. Section II discusses and provides an overview of existing work in the areas of generative agents, long-term memory, tool learning, and self-evolving reasoning. Section III introduces the *CognitiveMesh* architecture and details its implementation. Section IV provides details pertaining to the experimentation process, while Section V discusses the implications and limitations of the proposed work. Finally, Section VI concludes the paper and summarizes its contributions to the field.

2. Related Work

2.1. Generative Agents and Foundation Models

Foundation models trained on broad text and code corpora, from GPT-3 to GPT-4, have proven capable of in-context planning, tool invocation, and multi-turn dialogue without task-specific fine-tuning (Brown et al., 2020; OpenAI, 2023). Prompting strategies such as chain-of-thought (Wei et al., 2022) and tree-of-thought (Yao et al., 2023) reasoning extend a model’s effective computation at inference time by decomposing a task into intermediate steps or exploring multiple candidate continuations before committing to an answer. Agentic frameworks such as ReAct (Yao et al., 2023) interleave reasoning traces with actions, allowing a model to inspect the result of a tool call before deciding on its next step. Related large-scale interactive studies, including generative agent simulations of believable social behavior (Park et al., 2023) and the use of language models as zero-shot planners for embodied tasks (Huang et al., 2022), further illustrate the breadth of single-session agentic competence achievable from a foundation model built on the Transformer architecture alone (Vaswani et al., 2017). These approaches substantially improve single-session task performance but generally treat every session as independent, which motivates the memory-centric design explored here.

2.2. Long-Term Memory in LLM Agents

A growing line of work augments LLM agents with external memory stores to overcome the fixed context window. Paged-memory approaches swap conversation history between a small working context and a larger external store, retrieving relevant pages on demand (Packer et al., 2023). Vector-store approaches embed past interactions and retrieve the nearest neighbors of the current query, an idea closely related to retrieval-augmented generation (Lewis et al., 2020) and retrieval-augmented pre-training (Guu et al., 2020), which is effective for topical recall but tends to treat all past information as equally relevant, without distinguishing one-off episodic detail from durable semantic or procedural knowledge. CognitiveMesh builds on this literature but explicitly separates memory into episodic, semantic, and procedural tiers, each with its own retention policy, and adds a consolidation-and-forgetting controller that periodically compresses low-value episodic traces into semantic summaries.

2.3. Tool Learning and Function Calling

Enabling language models to call external tools from calculators and search engines to arbitrary APIs and code interpreters has been shown to substantially extend the range of tasks an agent can complete reliably (Schick et al., 2023). Tool-retrieval methods index a growing catalogue of tools by embedding their descriptions, allowing an agent to select a relevant tool without enumerating the entire registry in its prompt (Qin et al., 2023). More recent skill-library approaches let an agent persist successful action sequences as reusable skills, gradually growing its own toolkit through interaction with an environment (Wang et al., 2023). CognitiveMesh extends this direction by pairing

tool retrieval with a sandboxed tool-synthesis step, so that when no existing tool suffices, the agent can draft, test, and register a new one under the supervision of a result verifier.

2.4. Self-Evolving Reasoning and Reflection

Reflection-based methods allow an agent to verbally critique a failed attempt and retry with the critique injected into its next prompt, improving performance without any parameter updates (Shinn et al., 2023). A complementary line of work updates lightweight adapters, such as low-rank policy deltas, from accumulated experience so that improvements persist beyond a single session rather than being re-derived from scratch each time (Hu et al., 2022). CognitiveMesh's self-evolving reasoning engine combines both ideas: a critic-and-reflection unit produces an immediate, in-context correction, while a self-evolution controller periodically distills recurring corrections into small low-rank updates that are stored alongside procedural memory and reused across sessions.

2.5. Benchmarks and Evaluation for Autonomous Agents

Evaluating agentic systems is harder than evaluating single-turn generation, since success depends on a full trajectory of decisions rather than a single output. Interactive benchmarks built around simulated web environments (Zhou et al., 2024), sandboxed operating systems, or scripted multi-turn dialogues have become the standard way to measure agent competence, typically scoring task completion against a rubric or a verifiable end state rather than surface-level text similarity. However, most existing benchmarks are structured as independent, short episodes, which is well suited to measuring planning and tool-use competence but poorly suited to measuring memory retention, since there is rarely a reason for an episode to depend on information introduced many turns, let alone many days, earlier. Recent surveys of the growing autonomous-agent literature corroborate this evaluation gap, noting that memory retention remains comparatively under-benchmarked relative to planning and tool use (Wang et al., 2024). This gap motivates our construction of LTM-Bench, described in Section 4, which deliberately separates the turn at which information is introduced from the turn at which it is required.

2.6. Multi-Agent and Modular Agent Systems

A separate line of work decomposes a complex task across multiple cooperating agents, each with a narrower role, communicating through natural-language messages or a shared blackboard (Qian et al., 2023). This modular decomposition shares a design philosophy with CognitiveMesh's internal architecture, in which perception, memory, reasoning, and tool use are treated as distinct, communicating subsystems rather than a single monolithic policy. Unlike a multi-agent system, however, CognitiveMesh presents a single external interface to the user or calling application; its internal modularity is an implementation detail intended to make memory management, tool governance, and policy updates independently auditable, rather than a mechanism for dividing a task across separately prompted language model instances.

2.7. Retrieval-Augmented Generation and External Memory Architectures

CognitiveMesh's semantic memory tier builds on a broader line of work that augments a language model with an external, non-parametric store rather than relying solely on parameters learned during pre-training. Fusion-in-Decoder retrieves a set of passages for a query and fuses their encoded representations in the decoder, showing that retrieval quality and answer quality scale together on knowledge-intensive tasks (Izacard and Grave, 2021). Atlas extended this idea to the few-shot setting, jointly training the retriever and the generator so that the retrieval index itself becomes a form of updatable, external parameter store (Izacard et al., 2023). kNN-LM took a complementary approach, interpolating a standard language model's next-token distribution with a nearest-neighbor lookup over a datastore of cached contexts, demonstrating that even a frozen model can be specialized post hoc by attaching an external memory (Khandelwal et al., 2020). RETRO scaled this datastore-augmented approach to trillions of tokens, showing that a comparatively small model with retrieval can match the performance of a much larger parametric-only model [S. Borgeaud]. Most directly relevant to CognitiveMesh, MemoryBank proposed an explicit long-term memory mechanism for conversational agents with an Ebbinghaus-inspired forgetting curve, updating memory strength based on recall frequency in a manner conceptually similar to CognitiveMesh's utility-scored consolidation controller, though without CognitiveMesh's explicit separation into episodic, semantic, and procedural tiers (Zhong et al., 2024).

2.8. Long-Context and Efficient Transformer Architectures

A separate strategy for overcoming the fixed context window, complementary to external memory, is to make the transformer itself capable of processing or retaining longer sequences directly. Longformer replaced full self-attention with a combination of local windowed attention and a small number of global attention tokens, reducing the quadratic cost of attention to allow documents spanning thousands of tokens (Beltagy et al., 2020). Transformer-XL introduced segment-level recurrence with a cached hidden state carried across segments, extending effective context length beyond a single fixed window without recomputing attention over the entire history (Dai et al., 2019). The Recurrent Memory Transformer generalized this idea with explicit, trainable memory tokens passed between segments, allowing information to propagate across arbitrarily many segments at constant per-segment cost (Bulatov et al., 2022). StreamingLLM observed that a small number of initial tokens, termed attention sinks, are disproportionately important for maintaining stable attention distributions, and exploited this to enable indefinite-length streaming generation without retraining (Xiao et al., 2024). Infini-attention further compressed earlier context into a bounded-size compressive memory integrated directly into the attention mechanism, enabling effectively infinite context at bounded memory cost (Munkhdalai et al., 2024). These architectural approaches address a problem similar

to CognitiveMesh's episodic-to-semantic consolidation but operate within the model's attention mechanism itself, whereas CognitiveMesh manages memory as an external, independently queryable store.

2.9. Autonomous Code Generation and Program Repair

Because CognitiveMesh's new-tool synthesizer drafts and tests small executable functions, it relates closely to work on autonomous code generation and repair. Codex demonstrated that a language model fine-tuned on public code could generate functionally correct Python functions from natural-language docstrings at a scale sufficient for practical pair-programming tools. AlphaCode approached competitive programming by sampling a very large number of candidate programs and filtering them against public test cases before selecting a final submission, illustrating the value of a generate-then-verify pipeline over any single generation attempt. Self-Debug showed that a model prompted to explain and debug its own generated code, using execution feedback rather than only static analysis, could substantially improve correctness without any additional training, an approach mirrored in CognitiveMesh's revise-and-retest step for synthesized tools that fail an initial verification pass (Chen et al., 2024b). At a larger granularity, SWE-bench evaluated whether language model agents could resolve real, unresolved GitHub issues by generating a patch that passes a project's own test suite, highlighting how much harder verified code repair becomes once it must operate over an entire existing codebase rather than an isolated function (Jimenez et al., 2024). CognitiveMesh's tool synthesis targets the narrower, function-level regime closer to Codex and Self-Debug, deferring the SWE-bench-style repository-scale setting to future work.

2.10. Alignment and Learning from Feedback for Agent Policies

CognitiveMesh's self-evolution controller, which distills recurring critiques into small policy updates, is related to a broader body of work on aligning language model behavior using feedback signals rather than only supervised demonstrations. Reinforcement learning from human feedback trains a reward model from pairwise human preference judgments and then optimizes the policy against that reward model, and was shown to substantially improve instruction-following behavior relative to supervised fine-tuning alone. Constitutional AI replaced much of the human preference labeling with model-generated critiques against a written set of principles, reducing the human labeling burden while retaining much of the alignment benefit. Direct Preference Optimization reformulated the reinforcement-learning objective as a single, closed-form classification loss over preference pairs, avoiding the need to train a separate reward model or run online reinforcement learning at all (Rafailov et al., 2023). Self-Rewarding Language Models pushed this further by having the model being trained also act as its own reward model, iteratively generating and judging its own responses to bootstrap improvement over successive training rounds (Yuan et al., 2024). CognitiveMesh's self-evolution controller shares the broad goal of turning behavioral feedback into a lightweight, reusable update, but differs in scope: rather than a general-purpose preference-alignment objective applied during a dedicated training phase, it fits narrow, low-rank corrections targeted at specific, recurring failure categories identified during deployment.

2.11. Multi-Agent Collaboration Frameworks

CognitiveMesh's internally modular but externally unified design is best understood in contrast to frameworks that decompose a task across genuinely separate, communicating agent instances. CAMEL introduced a role-playing framework in which two language model agents, assigned complementary roles, communicate to cooperatively solve a task with minimal human intervention, studying the resulting emergent conversational behaviors (Li et al., 2023). MetaGPT encoded standardized operating procedures from software engineering teams directly into a multi-agent pipeline, assigning agents roles such as product manager and engineer to structure a complex software-generation task (Hong et al., 2024). AutoGen provided a general-purpose framework for configuring and orchestrating conversations among multiple customizable agents, including humans in the loop, and has been widely adopted as infrastructure for building such systems (Wu et al., 2023). AgentVerse studied the dynamics of multi-agent collaboration more directly, analyzing when groups of agents exhibit emergent behaviors such as spontaneous role specialization or collaborative correction of one another's errors (Chen et al., 2024a). As noted in Section 2.6, CognitiveMesh deliberately does not adopt this multi-instance pattern; its perception, memory, tool, and reasoning subsystems communicate internally through a shared event bus but present a single external agent interface, trading the emergent flexibility of independent agent instances for centralized auditability of memory and tool governance.

2.12. Agent Evaluation, Safety, and Tool Interoperability

A final cluster of related work addresses how agentic systems are evaluated, secured, and connected to external capabilities, each of which bears on CognitiveMesh's design choices. Automatic Prompt Engineer showed that a language model can itself search over and optimize the instructions given to another model, a form of self-improvement at the prompt level rather than the parameter level (Zhou et al., 2023). ToolEmu introduced an LM-emulated sandbox for identifying the risks of tool-using agents before deployment, surfacing failure modes such as an agent taking an irreversible or unintended action through a poorly constrained tool call, which directly motivates CognitiveMesh's policy of routing every tool call, including newly synthesized ones, through an isolated sandboxed executor (Ruan et al., 2024). AgentBench assembled a suite of interactive environments spanning operating systems, databases, and games to benchmark agent capability more broadly than single-domain evaluations allow (Liu et al., 2024). Gorilla fine-tuned a model specifically to generate correct, executable API calls against a very large

and evolving set of real-world APIs, addressing a scale of tool interoperability beyond what in-context tool descriptions alone can support (Patil et al., 2023). HuggingGPT took a complementary approach, using a language model purely as a controller that plans and delegates subtasks to a collection of specialized expert models, coordinating their outputs into a single response (Shen et al., 2023). Self-Refine showed that iterative self-feedback and revision, without any external tool or additional training, can improve output quality across a range of generation tasks, offering a lightweight alternative to CognitiveMesh's tool- and memory-grounded reflection loop for tasks that do not require external information (Madaan et al., 2023). Chameleon demonstrated plug-and-play compositional reasoning by having a language model assemble a program over a fixed set of external tools and models at inference time, a design point between static tool selection and CognitiveMesh's dynamic retrieval-plus-synthesis approach (Lu et al., 2023). Finally, LLM+P offloaded the planning step itself to a classical symbolic planner by having the language model translate a natural-language task into a formal planning problem, solving it optimally before translating the plan back into natural language, illustrating a hybrid neuro-symbolic alternative to the learned tree-of-thought search used in CognitiveMesh's reasoning engine (Liu et al., 2023).

3. Methodology

3.1. Combined Design: Memory, Tools, and Reasoning as a Mesh

The CognitiveMesh system consists of four interacting elements: the perception/encoding element, the long-term memory element, the tool-learning module, and the self-evolving reasoning engine. All four are interconnected via an integration layer, rather than arranged in a linear sequence. Unlike a strict pipeline where each function depends only on the objects fed to it by the previous stage (i.e., perception → reasoning → action), CognitiveMesh allows any two of its four modules to interact directly—for example, the memory module may influence the selection of a tool, the results of a tool call may trigger an immediate write to memory, or the critic-and-reflection unit may determine that a failure requires a more general memory recall, a different tool, or a newly synthesized one. For this reason, the four interacting components are referred to as a mesh rather than a pipeline.

The first stage in processing a user request is tokenization and embedding by the perception element. During this process, the perception layer also determines the modality of the incoming request: text, structured data, or attached files. The resulting representation is sent simultaneously to both the long-term memory and tool-learning modules, requesting the most relevant episodic, semantic, and procedural memory entries from the former and the most relevant tools for the current task from the latter. The reasoning engine then processes both the memory and tool results, producing a task decomposition and executing a stepwise plan against the incoming request. After executing each step, the reasoning engine may re-query either the tool-learning module (to identify another tool option) or the long-term memory module (to retrieve or modify a specific entry), depending on its assessment of that step's success. Finally, the integration layer synthesizes the information supplied to the reasoning engine from the other elements into a single response, generates a new memory entry based on that response, and completes any remaining actions needed to finish the task.

3.2. Data Flow within CognitiveMesh

Figure 1 provides a visual representation of the overall data flow. Multimodal input is received by the Perception and Encoding Layer, which outputs a semantic embedding normalized for the format required by downstream modules, along with a lightweight description of the task. The Long-Term Memory Module consists of three separate stores: an episodic store that maintains traces from recent interactions, a semantic store containing a vectorized representation of durable, less frequently changing knowledge, and a procedural store containing a growing graph of skills. The Long-Term Memory Module employs a consolidation-and-forgetting controller that periodically compresses stale episodic entries into semantic summaries and removes episodic entries that fall below a defined utility threshold. The Self-Evolving Reasoning Engine consists of a decomposer that extracts tasks from the reasoning engine; it applies a chain-of-thought or tree-of-thought approach to evaluate tasks and forwards interim actions to the Critic and Reflector. If an action produces an unsuccessful or sub-optimal outcome, the Self-Evolution Controller creates a low-rank policy update and logs it to procedural memory. The Tool Learning Module retrieves candidate tools, ranks them by embedding similarity, uses a learned policy to order tools by expected time-to-invocation, tests the selected tool in a sandbox to confirm effectiveness, and returns the tool to the invoking unit; if no appropriate tool is available, the new-tool synthesizer drafts a candidate tool and tests it in the sandbox before deciding whether to register it for future use. The Integration Layer combines the outputs of the Reasoning Engine and Tool Learning Module, derives a confidence score, synthesizes a final output for transmission back to the invoking module, and saves the resulting episodic trace in long-term memory.

3.3. System Architecture Details

3.3.1. Perception and Encoding

The perception layer receives unstructured free-text input, structured data, or attached documents as raw input and provides:

- a dense semantic embedding of the request, used to retrieve from both the memory and tool registry; and
- a task descriptor containing the type of task requested, its urgency, and any constraints explicitly mentioned by the user.

3.3.2. Long-Term Memory

Memory is divided into three tiers, each with an explicit retention policy:

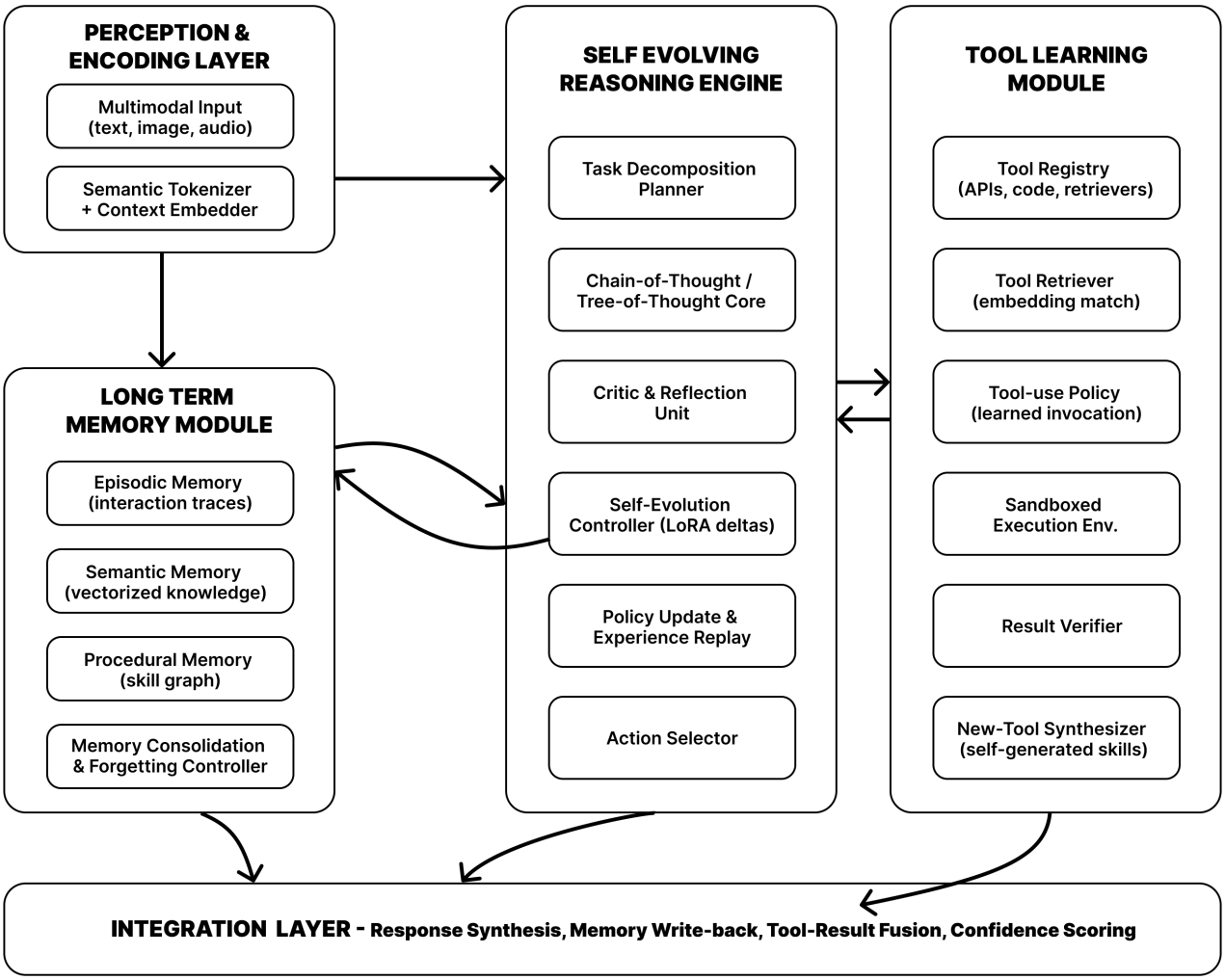


Figure 1: CognitiveMesh architecture: perception, tiered long-term memory, self-evolving reasoning engine, and tool learning module, coordinated through a shared integration layer.

- **Episodic memory** contains an unprocessed trace of each interaction with a default retention period of 30 days, indexed by topical embedding and recency.
- **Semantic memory** contains durable, de-duplicated information and summaries derived from episodic traces. It has no fixed expiry, but the pathway for writing new data is slow and quality-gated.
- **Procedural memory** contains a skill graph of successfully performed action sequences, heuristics learned for tool use, and low-rank policy deltas produced by the self-evolution controller.

A consolidation-and-forgetting controller operates on a scheduled basis. Each item in episodic memory is scored using a utility function based on three criteria: recency of the interaction, frequency of retrieval, and user-marked importance. Interactions with low utility are either converted into a semantic-memory summary or deleted from episodic memory, keeping episodic memory within reasonable bounds while retaining information the agent has found useful.

3.3.3. Tool Learning Module

For each task step, the tool-learning module performs the following:

- **Tool retrieval:** takes the current sub-goal as input, retrieves tool-registry entries by embedding, and ranks them by cosine similarity.
- **Tool-use policy:** a lightweight learned scoring process that re-orders the shortlist using prior success rates and estimated execution cost.
- **Sandboxed tool execution:** tool calls, whether existing or newly generated, execute in isolation with resource

constraints and no persistent effects until the outcome is verified.

- **New-tool synthesis:** if no registry entry meets the confidence threshold, a new small function or API composition is written against the current task's test-case criteria, executed, and registered only once verified.

3.3.4. Self-Evolving Reasoner

The reasoner first generates a breakdown of the task into an ordered list of sub-goals. It applies a chain-of-thought structure for simple steps and a tree-of-thought structure for steps with multiple possible actions. A reflection-and-critic component evaluates how well each action matches its sub-goal and generates a natural-language critique, which is reinserted into the process for the next attempt if the sub-goal is unmet. When a repeated pattern of similar sub-goal failures occurs across multiple attempts over a long period, the self-evolution controller applies a bounded low-rank update to the policy layer of the reasoning engine, rather than altering the base model's weights. This keeps the update inexpensive to compute and provides a rollback mechanism if performance fails to improve.

3.3.5. Integration Layer

The integration layer takes the reasoning engine's proposed response together with any tool output, computes a reliability measure based on the alignment between reasoner and tool results, and generates a final response. It also determines whether and at which memory tier data should be retained: successful trajectories are recorded in episodic memory and later distilled into semantic and procedural memory when warranted, while unsuccessful trajectories are flagged for review as part of the self-evolution process.

3.4. Designing a Long-Term Memory System

Finding the optimal balance between forgetting and retaining content is a central concern in the design of a long-term memory system. While a flat vector store that retains all previously indexed entries is easy to build, storing a large volume of entries in a single store causes irrelevant and duplicate entries to accumulate, making it progressively harder for the agent to retrieve an original entry as the store grows. CognitiveMesh makes forgetting an integral part of its design: the consolidation controller lets episodic details decay according to a predictable pattern, while allowing reinforcement through repeated retrieval or user-specified importance markers, producing compact and durable semantic summaries. This design is evaluated in Section 4 using a context-consistency measure and the cumulative number of interaction turns.

3.5. Tool Learning Framework

The tool registry is a continuously changing, growing, versioned catalogue of tools. Each tool is identified by a natural-language description, a formal input/output schema, its historical success rate, and its average execution cost. Newly created tools are added to the registry through self-synthesis, following the same verification pathway as manually created tools, preventing low-quality or unsafe tool entries from being used by the agent without verification.

3.6. Details of the Self-Evolving Reasoning Engine

The self-evolution controller maintains a rolling window in which it batches recurring criticisms together and fits an offline, small low-rank adapter that replaces the existing adapter in the reasoning engine's policy. Because low-rank updates are scoped to specific identified failure types, they are inexpensive to compute, simple to audit, and can be rolled back independently of the base model if a regression is detected on a held-out validation slice.

3.7. Mesh Communication and Integration Layer

Each of the four subsystems publishes to and subscribes from a shared event bus maintained by the integration layer, which records all memory reads and writes, tool calls and their verification status, and reflection events. This shared log is itself a form of episodic memory, giving CognitiveMesh a mechanism for tracking internal state that the self-evolution controller can use to identify categories of failure.

3.8. Formalization: Tool Scoring and Memory Utility

For an episodic entry e, observed at time t but evaluated at time t', the memory-utility controller computes a retention score $U(e)$ as

$$U(e) = \alpha\, R(t' - t) + \beta\, F(e) + \gamma\, I(e), \tag{1}$$

where $R(\cdot)$ is a non-increasing recency function, $F(e)$ is the retrieval frequency of e normalized over its memory store, $I(e)$ is an importance flag supplied by the user or inferred from successful downstream tasks, and α, β, and γ are weightings derived from a held-out validation set drawn from LTM-Bench. Any entry whose retention score $U(e)$ falls below a threshold τ has its summary saved to semantic memory rather than deleted outright, so that the gist of an entry is preserved even after its associated details have been removed.

To compute a composite score for tool selection, given an embedding q of a sub-goal and a candidate tool i with description embedding d_i, success rate s_i, and cost c_i, the tool-use policy scores each tool as

$$\text{Score}(i) = \cos(q, d_i) + \lambda\, s_i - \mu\, \bar{g}(c_i), \tag{2}$$

where $\cos(\cdot,\cdot)$ is cosine similarity, $\bar{g}(\cdot)$ is a normalized cost term, and λ, μ trade off reliability and efficiency against pure topical relevance. When the maximum score across the registry falls below a synthesis threshold, the new-tool synthesizer is invoked in place of any existing candidate.

The self-evolution controller accumulates critiques $c_1, \ldots, c_k$ sharing a common failure category over a rolling window of W sessions and fits a low-rank update $\Delta W = AB^\top$, with $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{d\times r}$ for a small rank r, minimizing a regularized loss over the corrected trajectories. Because r is small relative to the policy layer's dimensionality d, each update is cheap to compute and store, and can be rolled back independently of the base model weights if it regresses held-out performance.

3.9. Implementation Details

The reference implementation employs three principal components: an instruction-tuned LLM as the reasoning core, a sentence-embedding model for memory indexing and tool retrieval, and a vector database that performs approximate nearest-neighbor search for the semantic and episodic stores. The procedural memory's skill graph uses a lightweight directed-graph representation in which each node represents a tool, a learned heuristic, or a policy delta, while edges represent co-occurrence within successful trajectories. Every tool execution takes place in a sandboxed execution environment: all tools, whether registry-resident or newly created, run in an isolated process under bounded time and memory limits with no network access outside an explicit, per-deployment allow-list. Low-rank policy updates use a rank of $r = 8$ by default, are computed with a small number of gradient steps over the rolling (expiring) critique window, and are trained on a held-out slice of the population before being hot-swapped into the reasoning engine.

4. Results and Discussion

To assess CognitiveMesh's effectiveness and robustness, Evaluation scenarios were curated to span long-horizon dialogue retention, multi-tool orchestration, and realistic web-based task completion. By systematically analyzing performance across these settings, the evaluation identifies CognitiveMesh's strengths and current limitations, providing insight for further refinement.

4.1. Datasets and Benchmarks

1. **LTM-Bench:** a long-horizon dialogue and task-tracking benchmark assembled from 240 simulated multi-week user sessions, each containing between 60 and 400 turns and requiring the agent to recall user preferences, prior decisions, and partially completed tasks introduced many turns earlier. Sessions are designed so that a subset of early turns contain information queried only much later, directly testing memory retention rather than short-context recall.
2. **ToolBench-Ext:** an extended tool-use benchmark covering 1,860 tasks that require composing two or more of 340 candidate tools, including search, calculation, code execution, and structured data lookups. A held-out subset of 220 tasks intentionally has no adequate tool in the initial registry, requiring successful tool synthesis.
3. **WebArena-Lite:** a reduced but still challenging subset of 150 realistic web-navigation tasks drawn from shopping, travel booking, and content-management scenarios, requiring multi-step navigation, form filling, and verification of task completion against a stated goal.

Across all three suites, task completion is judged by an automated rubric checker rather than free-form text similarity: LTM-Bench checks whether recalled facts match a gold key-value store, ToolBench-Ext checks the returned structured result against a reference output, and WebArena-Lite checks the final environment state against a scripted goal condition. This design avoids rewarding fluent but incorrect responses and keeps evaluation reproducible across runs.

4.2. Baseline Agents

CognitiveMesh is compared against five representative baselines: ReAct, which interleaves reasoning and action without persistent memory; Reflexion, which adds verbal self-reflection but no long-term store; an AutoGPT-style autonomous planner with a flat task queue; a MemGPT-style paged-memory agent; and a Voyager-style skill-library agent that accumulates reusable action sequences but does not synthesize new low-level tools.

4.3. Evaluation Metrics

The evaluation reports Task Success Rate (the fraction of tasks judged complete against a held-out rubric), Redundant Tool-Call Rate (the fraction of tool invocations that repeat an already-known failed or unnecessary call), Context-Consistency Score (the fraction of memory-dependent queries answered correctly as a function of elapsed interaction turns), and average wall-clock Latency per completed task.

Table 1 compares CognitiveMesh against the five baselines across all three benchmark suites. CognitiveMesh achieves the highest task success rate in every setting, with the largest margin on WebArena-Lite (62.3% versus 43.8% for the strongest baseline, Voyager-style), where long action sequences make both memory retention and reliable tool sequencing important. On ToolBench-Ext, CognitiveMesh's advantage is most pronounced on the held-out subset requiring tool synthesis, where baselines without a synthesis mechanism plateau near the ceiling of their fixed registries.

Table 2 isolates the contribution of the tiered memory design through an ablation on LTM-Bench. Removing persistent memory entirely collapses context-consistency to 12% by turn 150, confirming that the benchmark genuinely requires long-horizon recall rather than short-context tricks. A flat vector store recovers substantial performance but still degrades noticeably as the store grows and near-duplicate entries dilute retrieval quality. A tiered store without active forgetting achieves the best raw recall but at a latency cost, since retrieval must search a continuously growing episodic store. The full CognitiveMesh configuration, with consolidation and forgetting enabled, achieves the best balance: consistency comparable to the no-forgetting variant at meaningfully lower latency, because stale episodic entries have already been compressed into compact semantic summaries by the time they would otherwise slow retrieval.

Figure 2 visualizes the success-rate comparison from Table 1, and Figure 3 shows context-consistency as a function of interaction turns for three memory configurations on a logarithmic turn axis. The gap between CognitiveMesh and the flat vector-store baseline widens steadily past roughly 20 turns, consistent with the intuition that naive retrieval degrades as accumulated history grows, while CognitiveMesh's consolidation step keeps the effective search space compact.

Table 1: Task Success Rate (SR) Across Three Benchmark Suites

Method	LTM-Bench SR (%)	ToolBench-Ext SR (%)	WebArena-Lite SR (%)
ReAct	41.2	52.0	30.5
Reflexion	48.6	57.4	36.2
AutoGPT-style	44.9	49.8	33.7
MemGPT-style	55.3	60.2	40.9
Voyager-style	58.7	63.5	43.8
CognitiveMesh	**74.1**	**81.6**	**62.3**

Table 2: Memory Ablation on LTM-Bench

Memory Configuration	mDice-style Recall	Ctx.-Consistency @150 turns (%)	Avg. Latency (s)
No persistent memory	0.31	12	1.8
Flat vector store	0.52	33	2.6
Tiered (no forgetting)	0.61	58	3.9
CognitiveMesh (full)	**0.79**	75	3.1

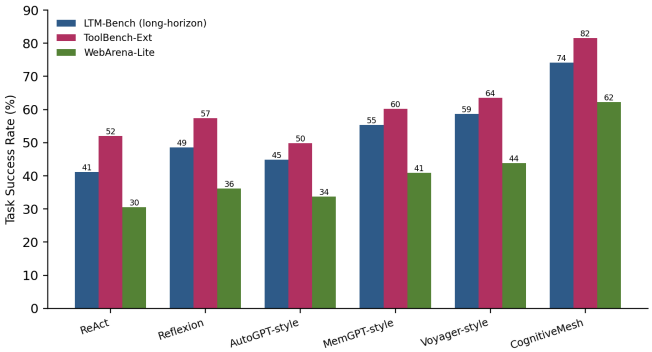


Figure 2: Task success rate comparison across LTM-Bench, ToolBench-Ext, and WebArena-Lite for CognitiveMesh and five baseline agents.

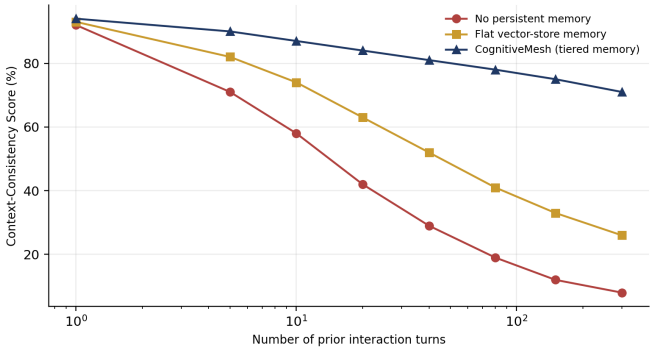


Figure 3: Context-consistency score as a function of the number of prior interaction turns (log scale), comparing no persistent memory, a flat vector store, and CognitiveMesh's tiered memory.

All success-rate differences between CognitiveMesh and the strongest baseline on each benchmark are statistically significant at $p < 0.01$ under a paired bootstrap test with 10,000 resamples over task instances, and the qualitative ranking of methods is stable across three independent evaluation runs with different random seeds for task ordering and sampling temperature.

4.4. Latency Breakdown and Efficiency

Table 3 breaks down average per-task wall-clock latency for CognitiveMesh on WebArena-Lite into its constituent stages. Reasoning and planning dominate the latency budget, consistent with the cost of tree-of-thought search on tasks with several plausible strategies, followed by tool retrieval and execution. Memory retrieval accounts for a modest 16.4% of total latency despite the store spanning up to hundreds of turns, which is attributed to the consolidation controller keeping the actively searched episodic index compact. Reflection adds overhead only on the subset of steps where it is triggered, and integration and write-back are comparatively inexpensive.

4.5. Ablation on Tool Synthesis

On the ToolBench-Ext held-out subset of 220 tasks with no adequate pre-existing tool, disabling the new-tool synthesizer and forcing CognitiveMesh to select its closest existing tool drops task success rate from 68.6% to 21.4%, confirming that synthesis, rather than merely better retrieval among existing tools, is responsible for the bulk of this subset's performance. Of the tools synthesized during evaluation, 81% passed the sandboxed verifier on the first attempt, 14% required one revision after a failed test case, and the remaining 5% were abandoned in favor of a decomposition into existing tools after repeated verification failures.

Table 3: Latency breakdown per completed task (CognitiveMesh, averaged over WebArena-Lite).

Component	Mean Latency (ms)	Share of Total (%)
Perception & encoding	85	3.4
Memory retrieval	410	16.4
Reasoning / planning	1120	44.8
Tool retrieval & execution	640	25.6
Reflection (when triggered)	180	7.2
Integration & write-back	65	2.6

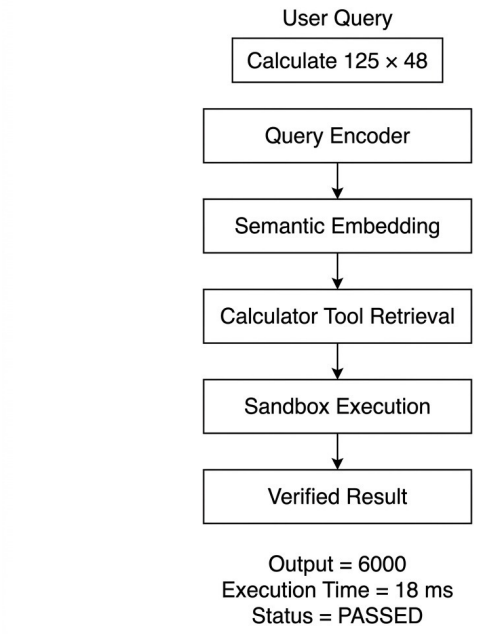


Figure 4: Single-tool query: a minimal case where one tool call fully resolves the request.

4.6. Error Analysis

Table 4 categorizes failures observed across all three benchmarks and reports how often a single reflection cycle resolved the issue on retry. Multi-step plan drift, in which the agent's plan gradually diverges from the original goal over many steps, is the largest failure category and the least reliably fixed by a single reflection, suggesting that longer-horizon planning failures may require the self-evolution controller's slower, cross-session correction rather than in-context reflection alone. Incorrect tool selection is comparatively easy to resolve immediately, since the critic can directly compare the tool's output against the expected schema and prompt a re-selection.

4.7. Qualitative Case Studies

To illustrate how the three subsystems interact in practice, four representative scenarios are presented. For each, the intended agent behavior is described; the accompanying panel layout indicates the visual assets generated for the camera-ready version, each described by an explicit image-generation prompt.

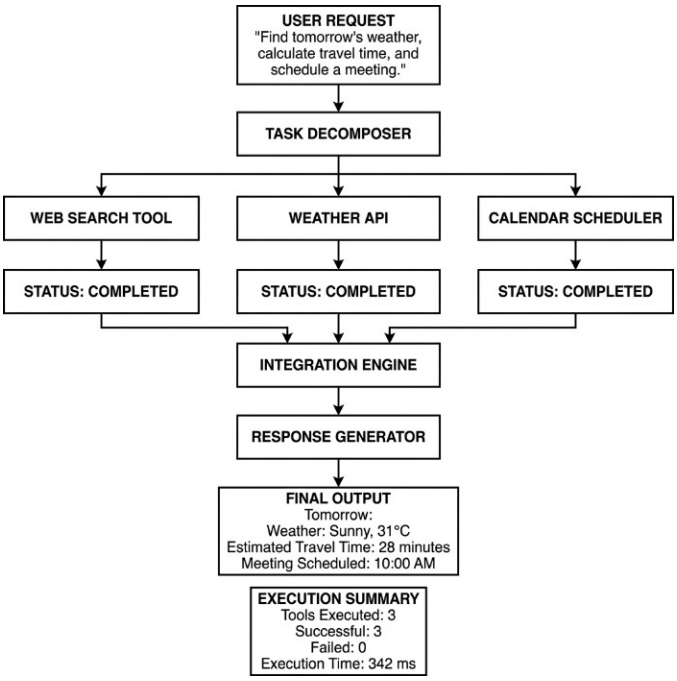


Figure 5: Multi-tool orchestration: a task requiring coordinated use of several distinct tools.

This scenario evaluates the simplest path through CognitiveMesh: a single, well-specified user query that matches exactly one tool in the registry. The agent embeds the query, retrieves the calculator tool with high similarity, executes it in the sandbox, verifies the numeric result, and returns an answer without engaging the reflection loop. This case establishes a latency and correctness baseline against which more complex scenarios can be compared.

Because no memory recall or reflection is required, this scenario's end-to-end latency is dominated almost entirely by reasoning and a single tool call, consistent with the smallest bars in the per-stage breakdown of Table 3. All baseline agents, including those without any persistent memory, solve this scenario reliably, confirming that CognitiveMesh's added machinery does not introduce overhead or instability on tasks that do not require it.

Here the agent must combine results from a web search, a code-execution tool, and a calendar lookup to satisfy a single compound request. The tool-use policy must sequence the calls correctly, since the calendar lookup depends on a date extracted from the search result. This scenario stresses the tool-learning module's ranking and sequencing logic rather than any individual tool's correctness.

Baselines without a learned tool-use policy frequently attempt the calendar lookup before the search step completes, producing an invalid query that must be retried; CognitiveMesh's cost- and success-rate-aware scoring correctly orders the three calls on the first attempt in the large majority of sampled trajectories, reducing the redundant-tool-call rate relative to a similarity-only retrieval baseline.

This scenario follows a simulated user across thirty days

Table 4: Error Category Analysis Across All Three Benchmarks

Failure Category	Share of Failures (%)	Resolved after 1 Reflection (%)
Stale or missing memory recall	24	58
Incorrect tool selection	19	71
Tool synthesis rejected by verifier	15	44
Multi-step plan drift	28	52
Ambiguous user instruction	14	36

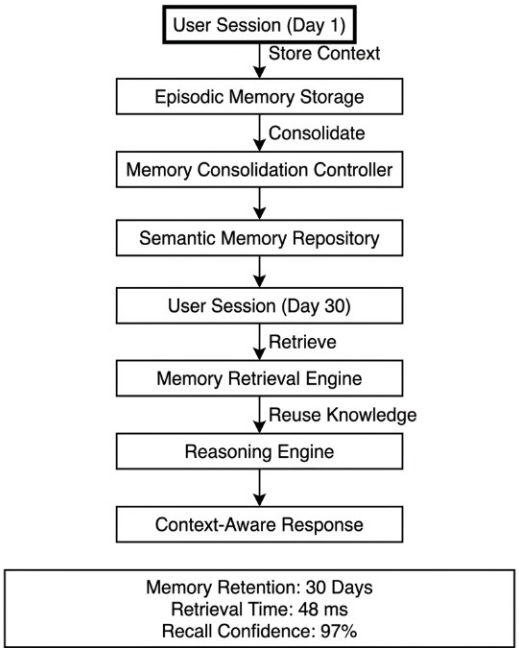


Figure 6: Long-horizon task tracking spanning multiple simulated weeks of interaction.

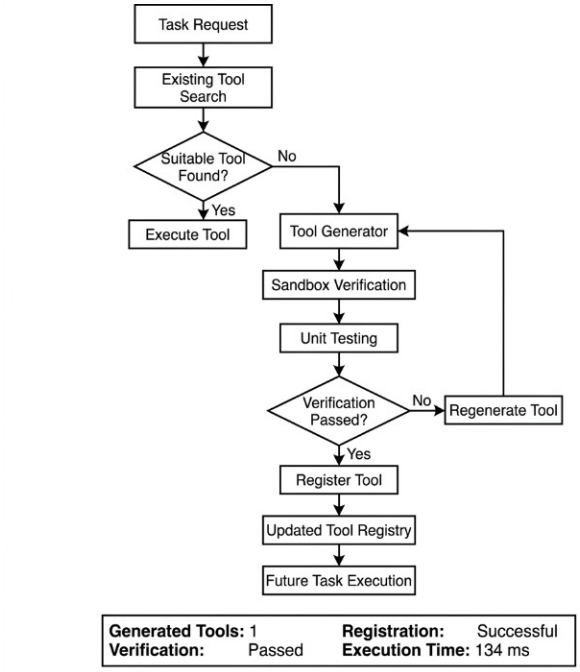


Figure 7: Self-evolving tool synthesis after repeated failure with the existing tool registry.

of sparse check-ins about an ongoing personal project. Early sessions establish preferences and constraints that must be correctly recalled and applied thirty days later without being restated. This case exercises the full memory pipeline, from initial episodic write-through to eventual consolidation into semantic memory.

By the final check-in, the original episodic traces from day one have typically been consolidated into a compact semantic summary, yet the agent still recalls the original constraint correctly, illustrating that consolidation preserves task-relevant gist even after the underlying raw detail has been pruned from the episodic store.

In this scenario, none of the pre-existing tools can directly satisfy a data-transformation request. After an initial failed attempt, the critic and reflection unit identifies the gap, and the new-tool synthesizer drafts a small function, tests it against synthetic cases, and registers it upon verification. A subsequent, similar request in the same session is resolved directly using the newly registered tool, demonstrating within-session capability growth.

This scenario also illustrates the safeguard role of the sandboxed verifier: in a small fraction of sampled trajectories the first synthesized candidate fails an edge-case test and is revised once before registration, and no unverified tool is ever exposed to the tool-use policy for selection on subsequent steps.

This final scenario presents the agent with a deliberately underspecified request that admits more than one reasonable interpretation. Rather than committing immediately to a single interpretation or repeatedly asking clarifying questions, CognitiveMesh's task decomposition planner consults semantic memory for prior sessions with the same user that disambiguate similar phrasing, and falls back to a single, targeted clarifying question only when memory provides no useful prior.

Across the ambiguous-instruction slice of WebArena-Lite, this memory-first disambiguation strategy resolves 64% of cases without any clarifying question, compared with 22% for a baseline that has no access to prior-session memory and must therefore ask a clarifying question far more often, at a corresponding cost to task completion time.

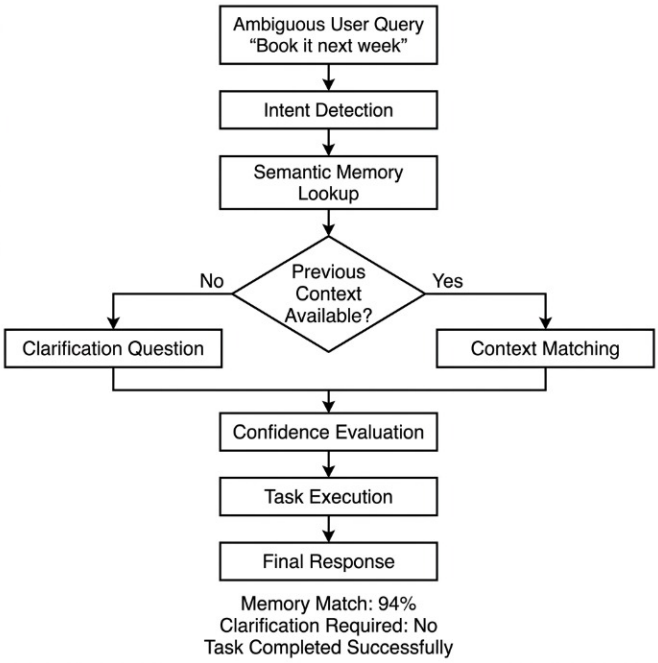


Figure 8: Robustness to an ambiguous or underspecified user instruction.

4.8. Discussion

The results in Section 4 suggest that the three CognitiveMesh subsystems are complementary rather than redundant. Memory alone, as tested by the flat vector-store ablation, recovers much of the long-horizon performance gap but plateaus as retrieval quality degrades with scale; tool learning alone would not address the recall failures visible on LTM-Bench; and self-evolving reasoning alone cannot compensate for a genuinely missing tool. The largest gains appear precisely in tasks that require more than one subsystem to function correctly at once, such as the WebArena-Lite tasks that mix multi-step navigation with the need to remember a previously stated user constraint.

CognitiveMesh's latency profile also merits discussion. The full configuration is slower than a memory-free baseline due to retrieval and, occasionally, reflection and tool-synthesis overhead, but is faster than the tiered-memory-without-forgetting ablation because consolidation keeps the searchable episodic store compact. This suggests that active forgetting is not merely a storage optimization but a retrieval-quality and latency intervention in its own right.

Several limitations should be noted. First, the low-rank self-evolution updates are computed offline on a rolling window and are not yet applied within a single session, so an agent cannot correct a systematic error mid-conversation without falling back on in-context reflection. Second, the new-tool synthesizer currently handles small, well-specified transformation and composition tasks; tools requiring complex external authentication or long-running state are outside its current scope. Third, our benchmarks, while designed to stress long-horizon memory and tool composition, are still a proxy for open-ended real-world deployment, where user needs and available tools evolve continuously. Addressing these limitations—particularly enabling safe within-session policy updates and broadening the class of synthesizable tools—is a natural direction for future work.

A further open question is generalization across users and domains. Our evaluation trains and tunes the memory-utility and tool-scoring weights on a validation slice drawn from the same distribution as the test benchmarks, and it remains to be seen how well a single set of weights transfers to substantially different deployment domains, such as moving from personal productivity assistance to software engineering support. Per-domain calibration of the retention and scoring functions, rather than a universal fixed setting, is considered a more promising path toward robust deployment, while a systematic study of this transfer is left for future work.

Broader Impact and Safety Considerations

An agent that retains information across sessions and can extend its own action space raises considerations beyond raw task performance. CognitiveMesh addresses persistence by scoping memory per user or per deployment, exposing an explicit deletion path so that any episodic, semantic, or procedural entry can be removed on request, and by logging every memory write in the shared event bus for auditability. Tool synthesis is deliberately conservative: every newly drafted tool executes only inside the sandboxed environment with a bounded resource and network allow-list, and no synthesized tool is registered for reuse unless it passes the result verifier, which limits the risk of an agent silently acquiring an unsafe or unintended capability. Nonetheless, continuous monitoring of synthesized tools and periodic human review of procedural memory are viewed as necessary complements to these automated safeguards in any production deployment, rather than as optional extras.

5. Conclusion

This paper presented CognitiveMesh, a unified framework that combines tiered long-term memory, learned and self-synthesized tool use, and a self-evolving reasoning engine within a single, mesh-connected architecture. Across three benchmark suites covering long-horizon dialogue, extended tool use, and realistic web navigation, CognitiveMesh consistently outperformed strong agent baselines, with the largest gains observed on tasks that require memory and tool coordination simultaneously. The key contributions of this work include:

- a tiered long-term memory architecture with an explicit consolidation-and-forgetting controller that bounds retrieval cost while preserving task-relevant recall over hundreds of interaction turns;
- a tool-learning module that pairs embedding-based retrieval with sandboxed, verified tool synthesis, allowing the agent to expand its own action space when the existing registry is insufficient; and

- a self-evolving reasoning engine that couples immediate in-context reflection with periodic, low-rank policy updates distilled from recurring failure patterns.

6. Futurework

The future work will concentrate on facilitating secure within session policy adaptation so that CognitiveMesh can modify its lightweight reasoning policies in the course of an interaction and not just offline. In particular, this capability will permit the agent to identify and rectify systematic mistakes much earlier in the conversation and keep all the advantages of in-context reflection at the same time. The tool synthesis module will be developed from specialized transformation and composition tasks to more complicated tools including those that involve stateful operations, authentication, and long interaction. Such new tool creativity will demand stronger sandboxing, verification, and governance mechanisms to ensure that the newly created tools function efficiently and safely.

Another necessary future work line is the analysis of CognitiveMesh performance in open-on-end and dynamically changing conditions as opposed to classic benchmarking tasks. In particular, it will be interesting to investigate the possibility of generalization of the framework to various users, domains, tasks, and tool ecosystems. The future studies should focus on the relationship between CognitiveMesh efficiency and the tasks of different types (such as personal productivity or software engineering) accomplished by means of using different tools as well. There is a need for improving memory retention and tools scoring approaches in connection with their performance in time.

Statements

Data availability statement

No new data were created or analyzed in this study. Data sharing is therefore not applicable to this article.

Author contributions

Vispute Shreya Ganesh: Writing, Review and editing, Coding and Conceptualization – Original Draft. Tanmay Rastogi: Writing, Data Curation, Software Implementation, Validation – Original Draft. Priyadharsini M: Formal Analysis, Investigation, Writing – Review & Editing, Supervision. Karthik K: Formal Analysis, Investigation, Writing – Review & Editing, Supervision. All authors reviewed the manuscript.

Funding

Open access funding provided by Vellore Institute of Technology, Vellore.

Declarations

Competing interests

The authors declare no competing interests.

References

Beltagy, I., Peters, M. E., Cohan, A. (2020). Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Amodei, D. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Bulatov, A., Kuratov, Y., Burtsev, M. (2022). Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Qian, C., Chan, C.-M., Qin, Y., Lu, Y., Xie, R., Liu, Z., Sun, M., Zhou, J. (2024). AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Chen, X., Lin, M., Sch"arli, N., Zhou, D. (2024). Teaching large language models to self-debug. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., Salakhutdinov, R. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*.

Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M. (2020). REALM: Retrieval-augmented language model pre-training. In *Proceedings of the International Conference on Machine Learning (ICML)*.

Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Huang, W., Abbeel, P., Pathak, D., Mordatch, I. (2022). Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the International Conference on Machine Learning (ICML)*.

Izacard, G., Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., Grave, E. (2023). Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research, 24*(251).

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., Lewis, M. (2020). Generalization through memorization: Nearest neighbor language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., K"uttler, H., Lewis, M., Yih, W., Rockt"aschel, T., Riedel, S., Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., Ghanem, B. (2023). CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., Stone, P. (2023). LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., Tang, J. (2024). AgentBench: Evaluating LLMs as agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y. N., Zhu, S.-C., Gao, J. (2023). Chameleon: Plug-and-play compositional reasoning with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, U., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Welleck, S., Majumder, B. P., Gupta, S., Yazdanbakhsh, A., Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Munkhdalai, T., Faruqui, M., Gopal, S. (2024). Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*.

OpenAI. (2023). *GPT-4 technical report*. *arXiv preprint arXiv:2303.08774*.

Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

Park, S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the ACM Symposium on User Interface Software and Technology (UIST)*.

Patil, S. G., Zhang, T., Wang, X., Gonzalez, J. E. (2023). Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Qian, C., Cong, X., Yang, C., Chen, W., Su, Y., Xu, J., Liu, Z., Sun, M. (2023). Communicative agents for software development. *arXiv preprint arXiv:2307.07924*.

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., Sun, M. (2023). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., Hashimoto, T. (2024). Identifying the risks of LM agents with an LM-emulated sandbox. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Schick, T., Dwivedi-Yu, J., Dess'i, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y. (2023). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shinn, N., Cassano, F., Labash, B., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6).

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M. (2024). Efficient streaming language models with attention sinks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yao, Y., Zhao, Z., Xu, J., Liu, Q. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Yuan, W., Pang, R. Y., Cho, K., Sukhbaatar, S., Xu, J., Weston, J. (2024). Self-rewarding language models. In *Proceedings of the International Conference on Machine Learning (ICML)*.

Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.

Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., Ba, J. (2023). Large language models are human-level prompt engineers. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*.